AnyBlox: Decoupling Storage Formats from Data Processing Systems

Liam Sanft
liam.sanft@mailbox.tu-dresden.de
TU Dresden
Germany, Dresden

Abstract

Open storage formats in data lakes have separated data from the systems that process it, but reading a format still requires a decoder written for that format inside every consuming system. With N systems and M formats, this yields $N \times M$ independent decoder implementations, which raises maintenance cost, increases the risk of security vulnerabilities, and discourages the adoption of new encodings. This paper introduces AnyBlox, a framework proposed by Gienieczko et al. (2025) that moves the decoder into the data. Each dataset ships with its own decoder as sandboxed WebAssembly bytecode, and any engine implementing a single scan interface can read the format without prior knowledge of it, reducing the integration burden to $N + M$.

1 Introduction

For decades, database systems operated as tightly coupled units in which storage format, processing logic, and data access were controlled by a single system. External systems submitted queries and received results, while everything in between remained internal concerns. With full control over the physical layout, systems could optimize both storage and processing of data freely within their own boundaries. As long as data resided within a single system, this model was sufficient [19].

However, the volume and variety of data collected has grown drastically. Data arrives continuously from a growing number of sources, in formats tailored to their respective domains rather than to any particular database system. As a result, the database landscape has evolved from a single system controlling all aspects of data management toward a heterogeneous, modular ecosystem of specialized systems, each handling a distinct part of the data pipeline and accessing a shared storage layer. Open file formats such as Apache Parquet[1] and Apache ORC[2], stored in data lakes [3], have gained wide adoption in this context. Beyond these, data scientists increasingly need to analyze data in domain-specific encodings from fields such as high-energy physics and bioinformatics [8].

Independently, research in data compression and encoding continues to produce formats that outperform established standards in storage efficiency and query throughput. These advances rarely reach production systems, however, because each system must independently implement support for every format it wishes to read [8].

Both trends converge on the same structural problem. Codd's principle of physical data independence [6], which assumes that a system controls its own storage format and shields applications from its physical details, no longer holds in a shared storage layer.

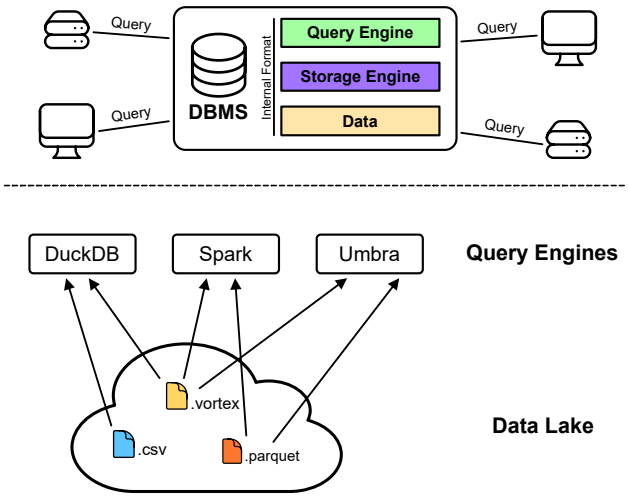

Figure 1: (Top) A monolithic DBMS controls its query engine, storage engine, and internal format; clients interact only through queries. (Bottom) In a data lake, multiple query engines access a shared pool of open file formats. Since each engine must implement a decoder for each format, N engines and M formats require $N \times M$ independent implementations.

In a shared storage layer, this assumption no longer holds. The format is determined by whoever produced the data. Every system that wants to read a given format must implement its own decoder. With N systems and M formats in active use, this results in $N \times M$ independent implementations to develop and maintain [8].

This results in higher implementation cost and maintenance effort, and simultaneously increases the risk of security vulnerabilities, as each implementation may contain bugs, as illustrated by a high-severity vulnerability in PostgreSQL's[3] JSON support that went undetected for over five years [17]. This initial hurdle discourages systems from supporting new formats altogether.

The consequence is a self-reinforcing cycle: a new format is not adopted by systems because its user base is too small to justify the implementation effort, yet its user base remains small precisely because it is not widely supported. Gienieczko et al. (2025) identify this as a core structural problem: an encoding must be popular enough to justify the cost of support, but will not gain popularity without being widely supported.

[1] Apache Parquet, https://parquet.apache.org (accessed: May 22, 2026)
[2] Apache ORC, https://orc.apache.org/ (accessed: May 22, 2026)
[3] PostgreSQL, https://www.postgresql.org/ (accessed: May 22, 2026)

This dynamic leads to format ossification: datasets continue to be produced using outdated format specifications solely to maintain compatibility with existing readers, even when more efficient encodings are available [12]. Evolution of a format is practically prevented, because any change risks breaking compatibility with the large number of systems that have independently implemented the old specification [8].

But what would be the solution for this combinatorial integration burden? An abstraction layer between storage formats and processing systems would reduce the $N \times M$ problem to $N+M$. Each system implements the interface once, and each format provides a single implementation against it. Gienieczko et al. (2025) propose exactly this and present AnyBlox as their realization.

2 Self-Decoding Data: The AnyBlox Design

A future-proof data access solution must satisfy two conceptual requirements simultaneously. First, it must introduce an abstraction layer that decouples format internals from processing systems and system internals from decoders, thereby reducing the $N \times M$ integration burden to $N + M$. Second, this abstraction must not preclude direct file access, since routing data through an intermediary process introduces transfer overhead that can dominate total runtime. These two goals are in tension: an abstraction layer must be located somewhere, and every candidate location either breaks direct access or reintroduces format-specific coupling [8].

Existing approaches each resolve this tension in favor of one side. Native integration preserves direct file access and achieves the best performance through tight coupling with host internals, but requires a separate decoder implementation per format per system, thus reproducing the $N \times M$ problem. Extension mechanisms reduce the per-format implementation effort, but each host exposes its own extension API, so a decoder written for system A cannot be reused in system B without a near-complete rewrite. Isolated extensions, such as containerized user-defined functions [18] or remote cloud user-defined code execution solutions provide both isolation and portability, but introduce a data transfer bottleneck between the host and the isolated decoder process [8].

Given these constraints, Gienieczko et al. (2025) argue that there is only one location for the decoder that satisfies both requirements: inside the data file itself. Packaging the decoder with the data eliminates the transfer bottleneck, since decoder and data reside in the same address space, while the abstraction is preserved because the host need not implement any format-specific logic. This is the core idea behind self-decoding data and the design premise of AnyBlox.

2.1 WebAssembly as the Decoder Runtime

A self-decoding dataset requires a decoder runtime that satisfies four criteria: **Portability**, **Security**, **Performance** and **Extensibility**.

Gienieczko et al. (2025) identify WebAssembly (Wasm) as a runtime that satisfies all four simultaneously. Wasm is a portable low-level bytecode and virtual machine specification, designed as an abstraction over modern hardware rather than a commitment to any specific programming model, making it language-, hardware-, and platform-independent [9].

2.1.1 Portability. A decoder implemented in Wasm runs on a wide variety of host architectures (e.g. browsers, mobile devices, embedded, and server workstations) [8]. For AnyBlox, this means a format author writes and compiles a decoder once, and any host can execute it via `libanyblox`, independent of the underlying hardware or operating system.

2.1.2 Security. The WebAssembly specification provides machine-verifiable memory safety and isolation guarantees [21]. AnyBlox reinforces this by being implemented predominantly in safe Rust, guaranteeing memory and thread safety. All unsafe code is confined to a single module handling Wasm memory maps, keeping it easily auditable [8].

2.1.3 Performance. Wasm is JIT-compiled to the host's native instruction set at runtime, making near-native performance attainable. Additionally, AnyBlox avoids unnecessary data copying through a custom memory management scheme. Gienieczko et al. (2025) note that AnyBlox can passively benefit from future improvements in Wasm compiler technology, since all JIT and sandboxing details are encapsulated in `libanyblox` and not exposed in the public API [8].

2.1.4 Extensibility. Since most general-purpose programming languages provide a Wasm toolchain, format authors can implement decoders in their language of choice without restrictions. The main practical obstacle is SIMD instructions: converting platform-specific SIMD intrinsics to Wasm's instruction set requires manual developer effort. However, since Wasm defines SIMD at the bytecode level, the resulting modules remain fully portable across architectures [8].

2.2 Reading a Dataset: From File to Query Pipeline

When a Host wants to read an AnyBlox dataset, it first opens the `.any` file and reads its metadata during query planning. The metadata provides the Host with the schema, the total number of rows, and a recommended batch size, all of which are needed to partition the workload across threads and schedule the scan before decoding begins.

The Host then hands the metadata and `.any` file to `libanyblox`, which extracts the Wasm Decoder module, JIT-compiles it, and caches the result. Subsequent uses of the same Decoder therefore incur no compilation cost. Following this, AnyBlox sets up the execution environment for the Decoder and returns a job handle to the Host.

This procedure describes standalone AnyBlox, where the dataset resides in its own `.any` file and the metadata is limited to schema, row count, and batch size. AnyBlox can alternatively be embedded into an existing host format such as Parquet, in which case the Host is the format's own decoding library and the embedded path inherits that format's statistics and row-group filtering. The decoding pipeline described below is identical in both cases. The two modes differ only in the metadata available to the Host, a distinction that becomes relevant for predicate pushdown (Section 3.3).

From this point, the Host repeatedly requests arbitrary tuple ranges from that handle. Modern query engines process data in batches to amortize operator communication overhead, so rather

than decoding the entire dataset at once, the Host issues successive calls for ranges of tuples [11]. AnyBlox delegates each such request into the sandboxed Wasm module and returns the decoded result to the Host. The sole function through which this happens is `decode_batch` [8]:

- `i32 data`: pointer to the start of the encoded data in Wasm linear memory
- `i32 data_length`: length of the encoded data in bytes
- `i32 start_tuple`: ID of the first tuple to decode
- `i32 tuple_count`: number of tuples to decode
- `i32 state`: pointer to the state page in Wasm linear memory
- `i64 projection_mask`: bitmask specifying which columns to decode, allowing the Host to skip columns that are not needed for the current query

The `data` parameter points to the encoded dataset inside the Decoder's execution environment, but how does the dataset get there in the first place? A naive solution would be to copy the entire dataset into a memory region accessible to the Decoder, but this would incur a heavy initialization cost proportional to the dataset size. Instead, AnyBlox maps the dataset file directly into the Decoder's linear memory via `mmap`, a system call that maps a file directly into the virtual address space of a process without copying it, so physical memory is only allocated when a page is first accessed.

Wasm's linear memory is a contiguous, byte-addressable memory region starting at address `0x0` that represents the entire address space accessible to the Decoder, isolated from the rest of the host process. AnyBlox controls its layout: the encoded dataset is mapped into it via `mmap`, and the state page is placed immediately after. The `data` parameter is therefore simply a pointer to the start of the mapped dataset within this region. AnyBlox reuses the linear memory region across successive jobs on the same thread, avoiding repeated initialization costs.

The `state` parameter points to a page within the linear memory that is zero-initialized at the start of every job. A Decoder can use it to cache metadata or memory allocations across successive `decode_batch` calls. For example, a run-end encoding Decoder that performs a binary search on the first call can store its current position on the state page and resume directly on the next call, skipping the search entirely. Because the state page resets at the start of each job, Decoders are idempotent: identical parameters on the same dataset always produce equivalent results.

Once decoding is complete, `decode_batch` returns a pointer to an Apache Arrow Record Batch within the linear memory. Arrow[4] is a columnar in-memory format developed by the Apache Software Foundation: rather than storing data row by row, it organizes each column as a contiguous array in memory, which allows query engines to process large amounts of data efficiently without unnecessary memory accesses. A Record Batch is a collection of such arrays, all of the same length, representing a horizontal slice of a table. Arrow standardizes the representation of common SQL types (e.g., integers, floating-point numbers, strings, dates, and more) and is supported by a wide range of query engines. The choice of Arrow as the uniform output format of AnyBlox follows the same principle as other intermediate representations in systems design, because by agreeing on a common language between two components, the number of required translations reduces from $N \times M$ to $N + M$. Every Decoder, regardless of the underlying encoding scheme, produces an Arrow Record Batch, and every Host that speaks Arrow can consume it without any AnyBlox-specific logic. AnyBlox translates the Wasm-internal pointer into a native address and passes the batch to the Host, where it enters the query pipeline. The degree of integration varies by system: DataFusion [13] uses Arrow as its primary internal representation throughout, making the transfer zero-copy end-to-end. DuckDB [16] and Spark[5] do not use Arrow internally, but both have built-in support for reading Arrow batches and converting them into their respective internal representations on the fly. This conversion logic is part of the mainline code of both systems and requires no AnyBlox-specific adaptation [8].

3 Discussion

The preceding sections introduced the AnyBlox architecture and its decoding model. Whether this design is practical depends on three questions:

(1) What overhead does the Wasm sandbox introduce?
(2) How much effort does integrating the framework into an engine require?
(3) Where does the approach reach its limits?

Gienieczko et al. (2025) report benchmarks and integration experience that cover both favorable and unfavorable cases for AnyBlox.

3.1 Performance Evaluation

Sandboxing a decoder inside Wasm adds a layer of indirection between the data and the query engine. Gienieczko et al. (2025) measure the resulting overhead directly. They run TPC-H at scale factor 20 in DataFusion [13] and vary the representation of `lineitem`, the largest table in the benchmark and therefore the one whose scan dominates the runtime, across three configurations:

Parquet read by DataFusion's native reader with Snappy[6] compression;
AnyBlox Vortex a Vortex file decoded inside the AnyBlox sandbox;
Native Vortex the same Vortex decoder compiled natively without any sandbox.

The first variant isolates the effect of the format; the latter two isolate the effect of the sandbox alone. Predicate pushdown was disabled throughout, so the numbers reflect scan and operator cost only. Vortex[7] is a recent columnar format.

The benchmark yields two results, summarized in Figure 2. Both Vortex variants achieve over 2× lower scan latency than Parquet while also reaching a better compression rate. For the Parquet reader, decoding accounts for nearly half of the processing time, while Vortex spends around 80% of its processing time in relational operators. The overhead of the sandbox itself is small. Sandboxed Vortex decoding adds 5% to the scan latency over native decoding, which amounts to a 1% increase in overall query latency and which

[4] Apache Arrow, https://arrow.apache.org/ (accessed: June 2, 2026)
[5] Apache Spark, https://spark.apache.org/ (accessed: June 2, 2026)
[6] Snappy, https://github.com/google/snappy/ (accessed: June 15, 2026)
[7] Vortex, https://vortex.dev/ (accessed: June 15, 2026)

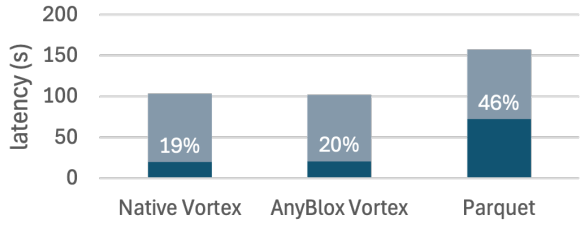

Figure 2: Total query latency (median, summed over 16 TPC-H queries at scale factor 20) in DataFusion, split into scan time (dark) and operator time (light). Percentages denote the scan share of total latency. Data source: [8].

the authors report as within measurement noise. Across this end-to-end workload, the cost of the sandbox is therefore negligible, and AnyBlox provides DataFusion with an encoding that outperforms its own native Parquet reader [8].

The microbenchmarks show where the overhead of the sandbox becomes visible. FSST [5] is a lightweight string compression scheme. Gienieczko et al. (2025) benchmark the decoding of three FSST-compressed string columns from the Taxpayer dataset of the Public BI Benchmark [7]. The 1.7 GB AnyBlox file expands into 2.5 GB of Arrow batches in memory. In this isolated scenario, the Wasm Decoder is over 43% slower than the same decoder compiled natively. The cause is architectural rather than incidental: the FSST decoder relies heavily on 8-byte reads, but Wasm uses a 32-bit address space by default, so each 8-byte read is emulated with two 4-byte reads. The authors describe this as a fundamental limitation of the current design. The 64-bit Wasm proposal would remove it, but it is incompatible with the software fault isolation mechanism on which AnyBlox's security guarantees rest, since that mechanism assumes a Decoder can address at most 8 GiB of memory [8].

Taken together, the benchmarks place the cost of the sandbox in perspective, though the evidence differs in strength across the two cases. For a complex decoder such as Vortex, the overhead is measured directly: in the end-to-end DataFusion workload it stays within measurement noise, and the more efficient encoding lets AnyBlox outperform the native Parquet reader. For the FSST worst case, the slowdown appears clearly in the microbenchmark, but its effect on full queries is not measured directly. Since the scan accounts for only about a fifth of processing time in the DataFusion workload, a decoding slowdown can affect only that share of the runtime. The microbenchmark therefore isolates the decoder under the conditions least favorable to it.

3.2 Integration Effort

The second claim AnyBlox makes is portability, meaning an engine can support the framework without a large implementation effort. Gienieczko et al. (2025) integrated AnyBlox into four systems spanning different execution paradigms (Umbra [15], DuckDB, DataFusion and Spark), all done by a single programmer with no prior experience in any of the host codebases. Table 1 lists the resulting effort. No integration required more than 1305 lines of code, and each one only added a new scan operator while leaving the rest of the engine, namely its optimizer, operators and execution layer, unchanged [8].

Table 1: Effort of integrating AnyBlox into each host system. Data source: [8].

System	Lines of code	Person-days
DataFusion	461	2
DuckDB	586	3
Spark	756	15
Umbra	1305	10

The authors caution that lines of code and integration time are not rigorous measures of software complexity, so these figures indicate rather than prove that the framework ports easily. With that caveat, they are the empirical counterpart to the $N \times M$ argument: a one-time integration lets an engine read any format providing a decoder.

3.3 Limitations

Two of AnyBlox's limitations follow from the design itself rather than from the current implementation. The first concerns predicate pushdown. Predicate pushdown evaluates filter conditions during the scan phase, so that rows failing a condition are discarded before they are decoded and passed to operators such as aggregation or joins. Decoding is computationally expensive, so prior filtering avoids processing data that would be discarded anyway [19].

Standalone AnyBlox provides no predicate pushdown API for row filtering. It supports projection pushdown, which restricts a scan to the requested columns, but not the filtering of individual rows. The reason is a deliberate design choice: filtering individual rows in a format-agnostic framework requires an equally format-agnostic and system-independent predicate expression format, which AnyBlox does not define. This reflects the same design philosophy visible elsewhere in the framework, that statistics and indexes should be decoupled from the storage format and managed by the query engine or a dedicated acceleration layer. The authors point to Substrait[8] as ongoing industry work toward a standardized predicate expression format, which could close this gap in a future revision [8].

This limitation is confined to standalone use. When an AnyBlox encoding is embedded into a host format such as Parquet at the row-group or column level, the host's existing row-group and column filtering continues to apply, so the embedded path retains the pushdown that the host already performs [8].

The second limitation concerns memory. Standalone AnyBlox maps the entire dataset into the Wasm linear memory region via `mmap` before decoding begins. The current implementation uses `wasm32`, which provides a 32-bit address space. Each mapped file is therefore limited to 4 GB. The authors note that file sizes in typical analytical deployments are commonly kept well below 4 GiB to facilitate cheap updates, so the restriction may have limited impact in practice. A migration to wasm64 would remove the limit, since Memory64 has been part of the WebAssembly standard since

[8] Substrait, https://substrait.io/ (accessed: June 15, 2026)

2025[9]. However, the performance impact of this migration in the context of AnyBlox is not yet studied, and wasm64 is incompatible with the software fault isolation mechanism on which AnyBlox's security guarantees rest. The `mmap` requirement also means that column projections cannot be pushed to remote files, because the entire dataset must reside in memory before the decoder is invoked, which is a relevant constraint in data lake deployments where data typically resides on object stores. Partial remote reads via page-fault handling are sketched as future work but not yet implemented [8].

4 Conclusion

Reading an unfamiliar storage format normally requires a reader written for that format inside every system that wants to consume it. AnyBlox removes this requirement by moving the reader into the data. A decoder travels with the dataset as sandboxed WebAssembly bytecode, and any engine that implements a single scan interface can decode the format without knowing it in advance. The cost of this design is the overhead of the sandbox, which the measurements of Gienieczko et al. (2025) show to be negligible for the end-to-end workloads tested.

The authors frame the underlying problem as an $N \times M$ problem: N systems each supporting M formats incur $N \times M$ implementation and maintenance effort [8]. An abstraction layer between the two sides reduces this to $N + M$, since each engine implements the interface once and each format provides one decoder. The authors draw this analogy explicitly from compiler construction, citing the language-independent intermediate representation of LLVM [14], which decouples M language frontends from N target backends through a shared representation. AnyBlox applies the same decoupling between storage formats and query engines. The comparison is an analogy rather than evidence: it motivates the design but does not establish that the result transfers from compilers to data formats.

This decoupling is not an isolated idea but part of a broader move toward composable database systems, away from large, monolithic designs. The authors place AnyBlox alongside several efforts that each modularise a different layer, among them a query engine built on composable design principles [13], execution-plan frameworks made extensible through sub-operators [4, 10], portable query optimizers [1, 2], and system-independent persistence mechanisms [20]. AnyBlox contributes the storage-format layer to this picture, and one of these efforts connects directly to it: the predicate pushdown limitation discussed in Section 3.3, where standalone AnyBlox deliberately defines no row-filtering predicate format, is a gap that Substrait, an emerging cross-engine standard for query plans, could close. The components are complementary, each standardising one interface between otherwise independent systems [8].

As a precedent that such modular frameworks can succeed, the authors invoke LLVM, which reshaped the design of compilers by establishing a shared, open representation [8, 14]. AnyBlox shows that the same principle could work for storage formats. Its evaluation demonstrates that a format-agnostic decoding layer can be built with low overhead and integrated into engines across different execution paradigms with modest effort. The LLVM comparison remains an analogy, and the evidence so far comes from a single evaluation by the original authors, so whether modular stacks come to displace integrated systems is a question that broader adoption and independent study will decide. What the paper establishes is that the decoding layer is no longer the obstacle.

[9]WebAssembly Memory64, https://github.com/WebAssembly/memory64 (accessed: June 15, 2026)

References

[1] Rana Alotaibi, Yuanyuan Tian, Stefan Grafberger, Jesús Camacho-Rodríguez, Nicolas Bruno, Brian Kroth, Sergiy Matusevych, Ashvin Agrawal, Mahesh Behera, Ashit Gosalia, Cesar Galindo-Legaria, Milind Joshi, Milan Potocnik, Beysim Sezgin, Xiaoyu Li, and Carlo Curino. 2024. Towards Query Optimizer as a Service (QOaaS) in a Unified LakeHouse Ecosystem: Can One QO Rule Them All? https://doi.org/10.48550/arXiv.2411.13704 arXiv:2411.13704 [cs.DB]

[2] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Prithviraj Pandian, Nikolay Laptev, and Ryan Marcus. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *Proceedings of the VLDB Endowment* 16, 12 (Aug. 2023), 3515–3527. https://doi.org/10.14778/3611540.3611544

[3] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. 2021. Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics. (2021).

[4] Maximilian Bandle and Jana Giceva. 2021. Database Technology for the Masses: Sub-Operators as First-Class Entities. *Proceedings of the VLDB Endowment* 14, 11 (July 2021), 2483–2490. https://doi.org/10.14778/3476249.3476296

[5] Peter Boncz, Thomas Neumann, and Viktor Leis. 2020. FSST: Fast Random Access String Compression. *Proceedings of the VLDB Endowment* 13, 12 (Aug. 2020), 2649–2661. https://doi.org/10.14778/3407790.3407851

[6] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (June 1970), 377–387. https://doi.org/10.1145/362384.362685

[7] Bogdan Ghit, Diego Tomé, and Peter Boncz. [n. d.]. White-Box Compression: Learning and Exploiting Compact Table Representations. ([n. d.]).

[8] Mateusz Gienieczko, Maximilian Kuschewski, Thomas Neumann, Viktor Leis, and Jana Giceva. 2025. AnyBlox: A Framework for Self-Decoding Datasets. *Proceedings of the VLDB Endowment* 18, 11 (July 2025), 4017–4031. https://doi.org/10.14778/3749646.3749672

[9] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. *SIGPLAN Not.* 52, 6 (June 2017), 185–200. https://doi.org/10.1145/3140587.3062363

[10] Michael Jungmair and Jana Giceva. 2023. Declarative Sub-Operators for Universal Data Processing. *Proceedings of the VLDB Endowment* 16, 11 (July 2023), 3461–3474. https://doi.org/10.14778/3611479.3611539

[11] Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, and Peter Boncz. 2018. Everything You Always Wanted to Know about Compiled and Vectorized Queries but Were Afraid to Ask. *Proceedings of the VLDB Endowment* 11, 13 (Sept. 2018), 2209–2222. https://doi.org/10.14778/3275366.3275370

[12] Laurens Kuiper. 2025. Query Engines: Gatekeepers of the Parquet File Format. https://duckdb.org/2025/01/22/parquet-encodings.html.

[13] Andrew Lamb, Yijie Shen, Daniël Heres, Jayjeet Chakraborty, Mehmet Ozan Kabak, Liang-Chi Hsieh, and Chao Sun. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data (SIGMOD '24)*. Association for Computing Machinery, New York, NY, USA, 5–17. https://doi.org/10.1145/3626246.3653368

[14] C. Lattner and V. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* IEEE, San Jose, CA, USA, 75–86. https://doi.org/10.1109/CGO.2004.1281665

[15] Thomas Neumann and Michael Freitag. [n. d.]. Umbra: A Disk-Based System with In-Memory Performance. ([n. d.]).

[16] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. https://doi.org/10.1145/3299869.3320212

[17] Red Hat, Inc. 2017. CVE-2017-15098. https://nvd.nist.gov/vuln/detail/CVE-2017-15098.

[18] Karla Saur, Tara Mirmira, Konstantinos Karanasos, and Jesús Camacho-Rodríguez. 2022. Containerized Execution of UDFs: An Experimental Evaluation. *Proceedings of the VLDB Endowment* 15, 11 (July 2022), 3158–3171. https://doi.org/10.14778/3551793.3551860

[19] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts* (seventh edition ed.). McGraw-Hill, New York, NY.

[20] Emil Tsalapatis, Ryan Hancock, Rakeeb Hossain, and Ali José Mashtizadeh. 2024. MemSnap μCheckpoints: A Data Single Level Store for Fearless Persistence. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3.* ACM, La Jolla CA

USA, 622–638. https://doi.org/10.1145/3620666.3651334

[21] Conrad Watt. 2018. Mechanising and Verifying the WebAssembly Specification. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2018)*. Association for Computing Machinery, New York, NY, USA, 53–65. https://doi.org/10.1145/3167082